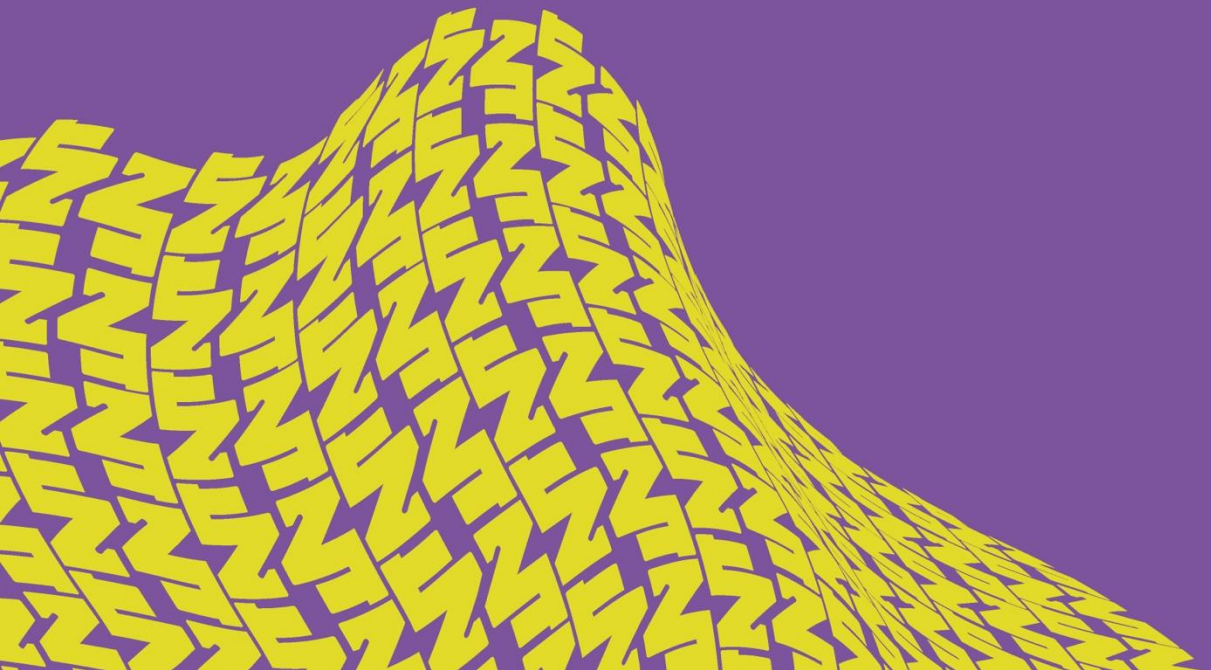


NT conference

21 – 23 September
2026



Regular Expressions in SQL Server 2025

Demystified RegEx

Tomaž Kaštrun, MVP

About



- Data scientist | DE & BI Developer
- SQL Server, SAS, R, Python, C#, SAP, SPSS
- 20+years experience MSSQL, DEV, BI, DM, DS
- Frequent community speaker, book author
- Avid coffee drinker & bicycle junkie

Session material:
https://github.com/tomaztk/SQLServer2025

http://tomaztsql.wordpress.com
tomaz.kastrun@gmail.com
@tomaz_tsql
/in/tomaztsql
http://github.com/tomaztk
https://mvp.microsoft.com/PublicProfile/5002196

Agenda

- RegEx refreshed
- The seven REGEXP functions — syntax and examples
- REGEXP vs. LIKE vs. PATINDEX vs. CLR
- Pattern complexity classes
- Backtracking & ReDoS — and why SQL Server 2025 is immune
- Engine algorithm benchmarks — NFA vs. DFA vs. RE2's compiled automaton
- Operation-type benchmarks — match, search, extract, replace, split, global
- Real-world use cases and benchmark guidance
- Table design and indexing

Disclaimer: PPT drawings were designed with the help of LLM ☺

What Is a Regular Expression?

A regular expression (**regex**) describes a **shape of text** — a pattern many different strings can match.

1. **Test**
   Does this text match the shape?
2. **Search**
   Where in the text does it match?
3. **Replace**
   Swap every match for something else.
4. **Split**
   Break text apart wherever it matches.

LIVE EXAMPLE

Find every price in a sentence:

`Total: $42.50 plus $6 tax`

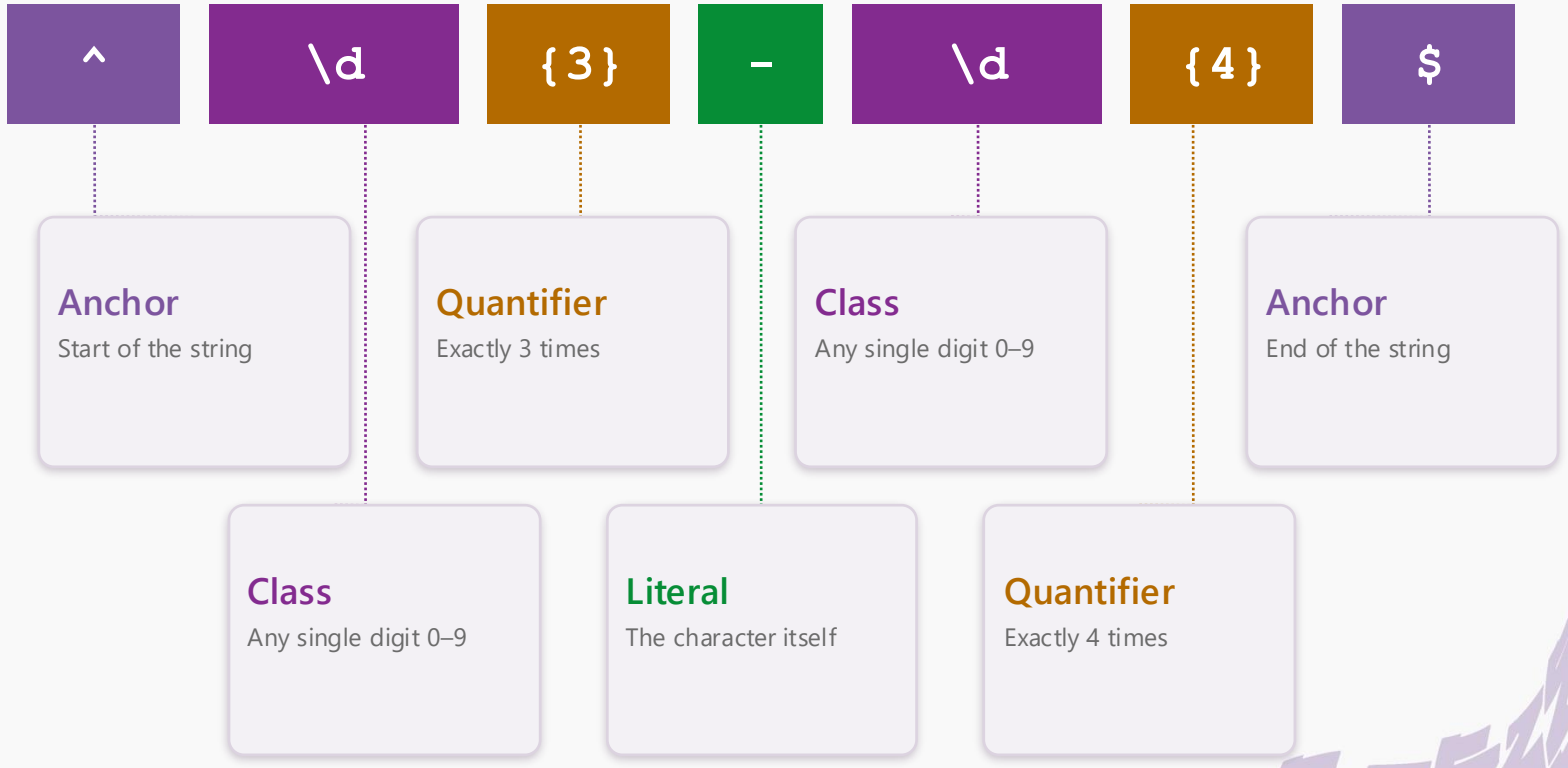
```
\$\d+(\.\d{2})?
```

Two matches were found — the engine keeps scanning after each one, so a single pattern catches every occurrence, not just the first.

GoTO: https://regex101.com/ (with regex flags /gm)
Type: "*Total: $42.50 plus $6 tax*"

Anatomy of a Regex

Every pattern is built from a few kinds of tokens — take this one apart piece by piece:



Character Classes

`\d`

Digit

0–9

4 2

`\w`

Word char

letters, digits, _

u s e r _ 1

`\s`

Whitespace

space, tab, newline

a ␣ b

`.`

Any character

except newline

c a t

`[abc]`

Set

one of a, b, c

c a b

`[^abc]`

Negated set

anything but a,b,c

x

`[a-z]`

Range

a through z

t e x t

`\D \W \S`

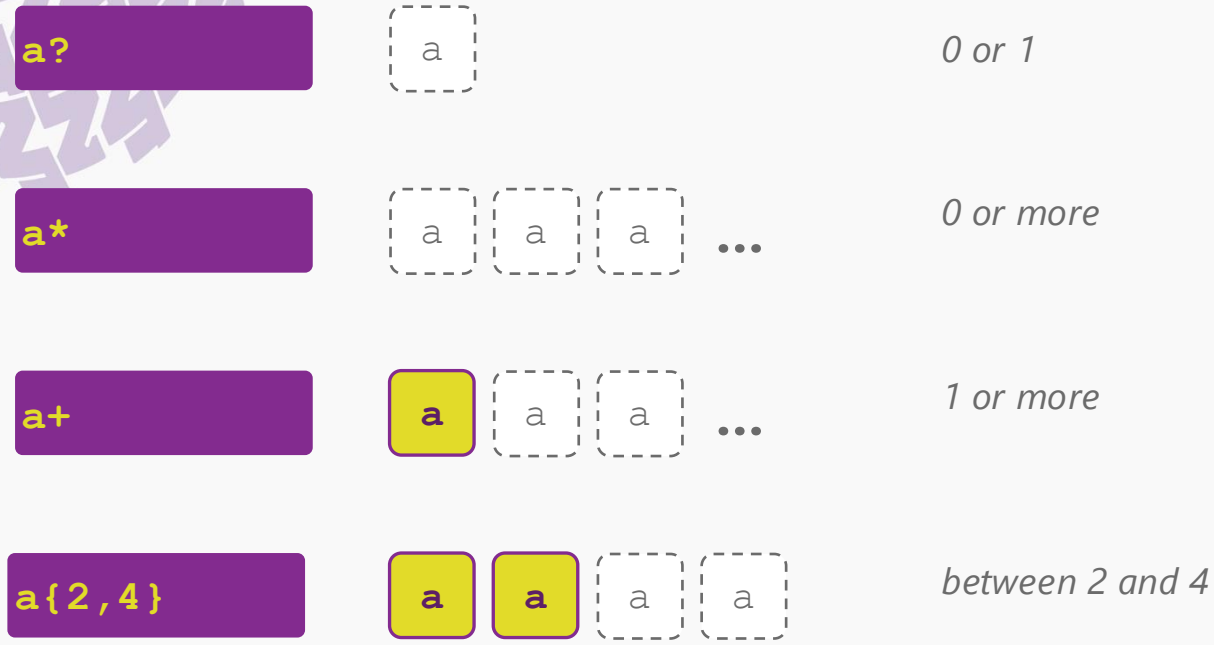
Negations

opposite of \d \w \s

a b c

Quantifiers

Quantifiers attach to the token on their left and control how many repetitions count as a match.



solid = required
dashed = optional
... = unlimited

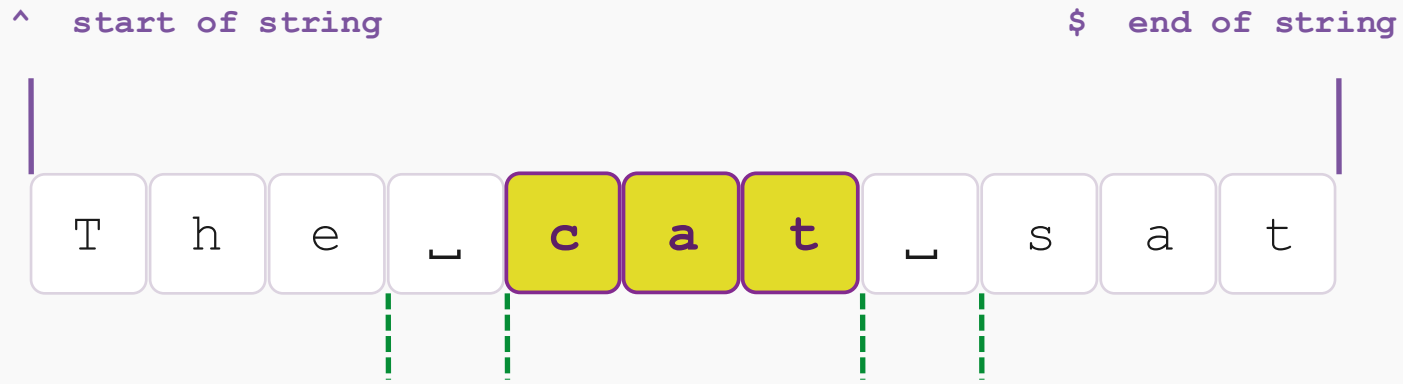
BY DEFAULT

Quantifiers are greedy — they grab as much text as possible, then give characters back only if the rest of the pattern needs them to.

Add a ? after any quantifier (a?) to make it lazy — match as little as possible instead.*

Anchors & Word Boundaries

Anchors don’t match a character — they match a position in the string.



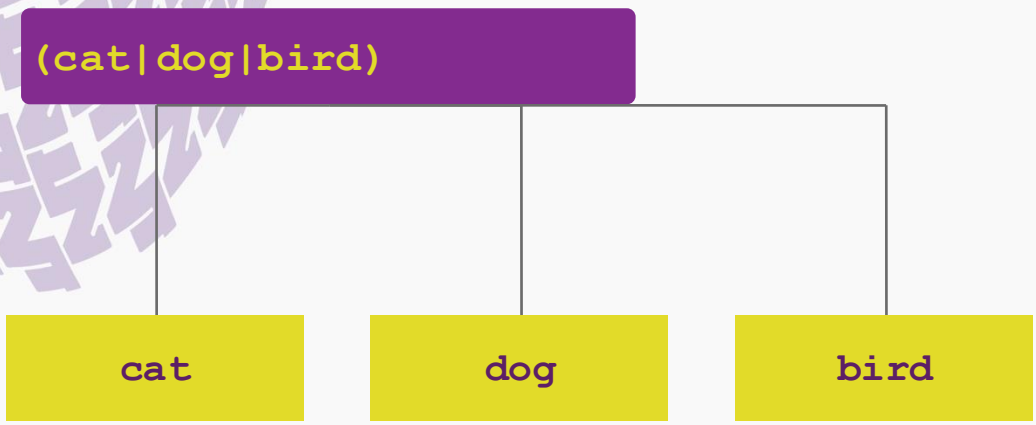
\b marks every edge between a word character and a non-word character (here: each space).

Pattern shown: `\bcat\b` — matches the word “cat” only as a whole word.

`^`	Start of string (or line, with the multiline flag)
`$`	End of string (or line, with the multiline flag)
`\b`	Boundary between a word char and a non-word char
`\B`	Anywhere that is NOT a word boundary

Groups, Alternation & Capturing

Alternation — match ONE of several options



Only one branch needs to match — the engine tries them left to right.

Capturing groups — pull pieces back out

```
(\d{4})-(\d{2})-(\d{2})
```

2	0	2	6	-	0	9	-	2	1

- **Group 1** → "2026"
- **Group 2** → "09"
- **Group 3** → "21"

`(?:...)` is a non-capturing group — it groups tokens for a quantifier or alternation without creating a numbered group to extract later.

Lookahead & Lookbehind

Lookaround checks what's next to a position but doesn't consume those characters as part of the match.

Lookahead

(?=…)



solid = consumed match · dashed = peeked only

matches "100" only if "px" follows

Negative lookahead

(?!…)



solid = consumed match · dashed = peeked only

matches "100" only if "px" does NOT follow

Lookbehind

(?<=…)



solid = consumed match · dashed = peeked only

matches "42" only if "$" comes before

Negative lookbehind

(?<!…)



solid = consumed match · dashed = peeked only

matches "42" only if "$" does NOT come before

Common Patterns Cheat Sheet

Email

```
^[\w.+-]+@[\w-]+\.[a-z]{2,}$
```

t o m @ n t k . s i

Phone format (according to A1) ☺

```
^\(\d{3}\)\s\d{3}-\d{3}$
```

(0 4 0) ␣ 4 7 4 - 5 1 4

URL

```
^https?://[\w.-]+\.\w+
```

h t t p s : / / a . i o / x

Date (ISO)

```
^\d{4}-\d{2}-\d{2}$
```

2 0 2 6 - 0 9 - 2 1

Hex color

```
^#[0-9A-Fa-f]{6}$
```

E 2 D B 2 9

IPv4

```
^(\d{1,3}\.){3}\d{1,3}$
```

1 9 2 . 1 6 8 . 1 . 1

Some of good practices

IS OK

- Anchor with ^ / $ when the whole string must match, not just part of it.
- Use an explicit negated class [^"]* instead of a lazy .*? when scanning up to a delimiter.
- Name or comment complex patterns — a regex is write-once, read-never by default.
- Test against edge cases: empty strings, very long strings, and Unicode input.

Meeeeh....

- Nested repeating groups like (a+)+ — they can cause catastrophic, exponential slowdowns.
- Greedy .* when you mean “up to the next X” — it can swallow far more than intended.
- Building a pattern from unsanitized user input without limits or testing.
- Assuming a pattern that works on ASCII samples behaves the same on accented or multi-byte text.

From Theory to Practice in SQL Server 2025

One Example, Three Approaches

The task: find every valid IPv4 address in a text column.

Approach	How it's done	Handles edge cases?	Speed at scale	Verdict
LIKE / PATINDEX	`Chain wildcard patterns, e.g. LIKE '[0-9]%.[0-9]%.[0-9]%.[0-9]%'`	No — can't bound 1–3 digits or reject 999.999.999.999	Slow — many chained scans	✗ **Unreliable**
LLM / AI model	Ask the model to judge each row, one call at a time	Inconsistent — non-deterministic, can hallucinate on edge cases	Very slow & costly — network round trip per row	✗ **Not fit for set-based SQL**
REGEXP_LIKE (SQL Server 2025)	`REGEXP_LIKE(col, '^(\d{1,3}\.){3}\d{1,3}$')`	Yes — exactly as specified, in one line	Fast — native, set-based, in-engine	✓ **Deterministic & free**

Same conclusion for phone numbers, dates, hex colors, log-line parsing — anywhere the shape of the text is precisely known.

Enabling Regex Support

Three of the seven functions require database compatibility level 170 or higher.

```
-- Check the current compatibility level
SELECT name, compatibility_level
FROM sys.databases
WHERE name = RegexBench';

-- Raise it to unlock REGEXP_LIKE, REGEXP_MATCHES,
-- and REGEXP_SPLIT_TO_TABLE
ALTER DATABASE RegexBench
SET COMPATIBILITY_LEVEL = 170;
```

Sample data

```
CREATE TABLE dbo.CustomerContact (
    ContactID     INT IDENTITY PRIMARY KEY,
    RawText       NVARCHAR(400) NOT NULL,
    -- e.g. 'Reach Jane Doe at jane.doe@contoso.com
    --       or +1 (555) 204-9981, ref #A-2291'
    LoggedAt      DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);
```

Why Native Regex, Why Now

- Regex functions reached General Availability in SQL Server 2025 and Azure SQL in November 2025
- Seven functions, POSIX-flavored, covering most of the PCRE/PCRE2 syntax developers already know
- **Built on RE2 — Google's regex engine, not the backtracking engine .NET or PCRE use**
  - RE2 guarantees execution time that is linear in input length — no catastrophic backtracking, no ReDoS
  - That guarantee is a trade-off, not a free lunch: RE2 deliberately omits backreferences and lookaround (more on this later)
- Replaces two previous workarounds: PATINDEX wildcards (limited) and CLR assemblies wrapping .NET regex (full-featured but requires CLR enabled and deployed)

Scalar Functions vs. Table-Valued Functions

Function	Category	Returns
REGEXP_LIKE	Scalar	bit — does the pattern match?
REGEXP_COUNT	Scalar	bigint — how many times does it match?
REGEXP_INSTR	Scalar	int — start/end position of a match
REGEXP_REPLACE	Scalar	string — source with matches substituted
REGEXP_SUBSTR	Scalar	string — the Nth captured substring
REGEXP_MATCHES	Table-valued	one row per match, with capture groups as JSON
REGEXP_SPLIT_TO_TABLE	Table-valued	one row per token, split on the pattern

Rule of thumb: the five scalar functions go in SELECT / WHERE; the two table-valued functions go in FROM.

REGEXP_LIKE — Boolean Match Test

REGEXP_LIKE(string_expression, pattern_expression [, flags]) → bit

```
SELECT ContactID, RawText
FROM dbo.CustomerContact
WHERE REGEXP_LIKE(RawText, '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}');
```

The regex equivalent of LIKE, but with full pattern power. See the indexing slide later for why this is expensive in a WHERE clause on large tables.

REGEXP_COUNT — Count Occurrences

REGEXP_COUNT(string_expression, pattern_expression [, start [, flags]]) → bigint

```
SELECT ContactID,
       REGEXP_COUNT(RawText, '[0-9]') AS DigitCount
FROM dbo.CustomerContact
WHERE REGEXP_COUNT(RawText, '[0-9]') >= 10;
```

Handy for quality checks — e.g. flagging rows whose phone-like sequences don't have enough digits.

REGEXP_INSTR — Locate a Match

REGEXP_INSTR(string_expr, pattern [, start [, occurrence [, return_opt [, flags [, group]]]]]) → int

```
SELECT ContactID,
       REGEXP_INSTR(RawText, '\+?[0-9][0-9 ().-]{7,}[0-9]') AS PhoneStart
FROM dbo.CustomerContact;
```

return_opt = 0 (default) gives the start position; return_opt = 1 gives the position just past the match — the same choice PATINDEX forces you to build yourself with LEN() and SUBSTRING().

REGEXP_REPLACE — Substitute Matches

REGEXP_REPLACE(string_expr, pattern, replacement [, start [, occurrence [, flags]]]) → string

```
SELECT ContactID,
       REGEXP_REPLACE(RawText,
           '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}',
           '[redacted-email]') AS Masked
FROM dbo.CustomerContact;
```

One call replaces what used to take PATINDEX + SUBSTRING + REPLACE chained together — and it's the fastest of the seven functions relative to its legacy equivalent (more on this in the benchmarks section).

REGEXP_SUBSTR — Extract a Match

REGEXP_SUBSTR(string_expr, pattern [, start [, occurrence [, flags [, group]]]]) → string

```
SELECT ContactID,
       REGEXP_SUBSTR(RawText, 'ref #([A-Z]-[0-9]{4})', 1, 1, '', 1) AS RefCode
FROM dbo.CustomerContact;
```

The final group argument pulls a specific capture group — group 1 here returns just A-2291, not the full 'ref #A-2291' match.

REGEXP_MATCHES — All Matches, As Rows

REGEXP_MATCHES(string_expression, pattern_expression [, flags]) → table

```
SELECT m.match_id, m.start_position, m.end_position,
       m.match_value, m.substring_matches
FROM dbo.CustomerContact c
CROSS APPLY REGEXP_MATCHES(c.RawText, '#([A-Za-z0-9_]+)') m
WHERE c.ContactID = 42;
```

Columns returned: match_id, start_position, end_position, match_value, and substring_matches (a JSON array of each capture group's value, start, and length). No match → no row, so CROSS APPLY (not OUTER APPLY) is usually right.

REGEXP_SPLIT_TO_TABLE — Tokenize on a Pattern

REGEXP_SPLIT_TO_TABLE(string_expression, pattern_expression [, flags]) → table(value, ordinal)

```
-- STRING_SPLIT can only split on a fixed delimiter;
-- this splits on *any* run of whitespace or commas
SELECT c.ContactID, t.value, t.ordinal
FROM dbo.CustomerContact c
CROSS APPLY REGEXP_SPLIT_TO_TABLE(c.RawText, '[\s,]+') t;
```

This is the regex-aware sibling of STRING_SPLIT. If the pattern never matches, the function returns the original string as a single row.

QUICK REFERENCE

Match Option Flags

Flag	Meaning	Default
c	Case-sensitive matching	true
i	Case-insensitive matching	false
m	Multi-line mode — ^ and $ match at line boundaries, not just string boundaries	false
s	"Single-line" / dot-all — . also matches \n	false

Flags combine in one string, e.g. 'im'. If contradictory flags are supplied (e.g. 'ic'), SQL Server uses the last one. An unsupported character raises an error naming the valid set: {c, i, s, m}.

REGEXP vs. LIKE vs. PATINDEX vs. CLR

Approach	Pattern power	Typical performance	When to reach for it
LIKE	Wildcards only: %, _, [ ]	Fastest — can seek an index	Simple prefix/suffix/contains checks
PATINDEX	Same wildcard set as LIKE, plus position	Fast, but scans	Need the position of a wildcard match
CLR (.NET regex)	Full .NET regex — backreferences, lookaround	Fast, but requires CLR enabled & deployed	You need a feature RE2 doesn't have
REGEXP_* (RE2)	POSIX / most of PCRE2, no backrefs or lookaround	Similar to legacy in most cases; slower for LIKE-equivalent filters	Complex patterns, extraction, or split — new code

None of these strictly dominates the others — pick based on the pattern complexity you actually need, not habit.

From Simple to Sophisticated

- Literal search — exact substring, no metacharacters
- Simple character classes — [A-Z], [0-9], \d, \w, \s
- Alternation — a|b|c, choosing between branches
- Anchored vs. unanchored — ^...$ pinning a match to string or line boundaries
- Unicode matching — classes and flags over non-ASCII text
- Back-references — matching text captured earlier in the same pattern
- Each step up this list costs more to evaluate — and, as we'll see, one of them isn't available at all in SQL Server 2025

Literal Search & Character Classes

Literal search

- No metacharacters — the pattern is the string
- 'ref #' matches only that exact text
- Cheapest possible pattern to evaluate — but LIKE '%ref #%' does the same job faster

Character classes

- [0-9] / \d — any digit
- [A-Za-z] / \w — letters (and \w includes digits + underscore)
- [^0-9] — negated class: anything but a digit
- \s — whitespace; [\s,]+ from the split example on slide 14

Alternation

Alternation tries branches left to right and takes the first one that matches at each position.

```
-- Match US or UK style phone prefixes
SELECT RawText
FROM dbo.CustomerContact
WHERE REGEXP_LIKE(RawText, '(\+1|\+44|0)[0-9 ()-]{7,}');

-- Grouping with (?:...) avoids creating a capture group
-- when you don't need REGEXP_SUBSTR / REGEXP_MATCHES to see it
WHERE REGEXP_LIKE(RawText, '(?:jane|john)\.doe@contoso\.com');
```

More branches mean more work per position in the worst case — keep alternations narrow and put the most likely branch first where it's safe to do so.

Anchored vs. Unanchored

Unanchored — default

- '[0-9]{4}' matches a 4-digit run anywhere in the string
- The engine tries every starting position until one succeeds
- Correct for "contains a match" questions

Anchored — ^ and $

- '^[0-9]{4}$' requires the entire string to be exactly 4 digits
- With the m flag, ^ / $ also match at line breaks, not just string start/end
- Anchoring early lets the engine reject non-matches faster — cheaper on rejects, not necessarily on matches

Unicode Matching

- REGEXP_* functions accept char/varchar and nchar/nvarchar — Unicode patterns and text both work
- \w, \d, and \s behave according to the pattern's semantics — test against your actual character set rather than assuming ASCII-only behavior
- **Performance callout, from real-world testing**
  - Unicode patterns showed the largest performance gap between legacy T-SQL functions and REGEXP_* in independent benchmarking
  - Treat Unicode (nvarchar) regex the same way you'd treat any nvarchar column: use it when the business need is real, not as a default

Back-References — Not Supported

A back-reference (\1, \2, …) matches whatever an earlier group captured. RE2 — the engine behind SQL Server 2025's regex functions — does not support them, by design.

```
-- This PCRE/.NET-style pattern finds doubled words
-- ('the the'). It will NOT work in REGEXP_LIKE:
--   \b(\w+)\s+\1\b

-- Back-references make matching provably require
-- backtracking — which is exactly what RE2 exists to avoid.
```

Same story for lookahead / lookbehind — also unsupported. If you truly need either, that's a signal to reach for the CLR regex path (slide 16), not to fight the engine.

Back-References — workournd

```
-- Split-and-compare in T-SQL —
-- use REGEXP_SPLIT_TO_TABLE + GROUP BY/HAVING COUNT(*) > 1 for the common
-- "find duplicate tokens" case

Fall back to a CLR-hosted .NET or PCRE function for the rare case where you truly need \1 — at the
cost of losing RE2's linear-time guarantee
```

Why SQL Server 2025 Doesn't Have a ReDoS Problem

- Classic regex engines (PCRE, .NET, Java, JavaScript, Python re) are backtracking NFAs
  - Certain patterns — nested quantifiers over ambiguous alternatives — can force exponential-time backtracking on the right (wrong) input
  - That's a Regular Expression Denial of Service (ReDoS): a small, well-formed pattern plus a crafted string that pegs CPU for seconds or minutes
- **SQL Server 2025 runs regex on RE2, which sidesteps the whole class of problem**
  - RE2 compiles patterns into a finite automaton and guarantees execution time linear in input length, no matter the pattern
  - The trade-off is the missing back-references and lookaround from the previous slide — RE2 gave those up specifically to keep the linear-time guarantee
- Practical upshot: there's no engine-level timeout to configure and no backtracking limit to tune — the questions the outline poses ("what input length times out?", "is there a backtracking limit?") don't apply here

NFA vs. DFA vs. RE2's Compiled Automaton

Approach	How it matches	Worst case	Used by
Backtracking NFA	Tries one path, backs up and retries on failure	Exponential on pathological patterns	.NET, PCRE, Java, Python re, JavaScript
Classic DFA	Precomputes every possible state up front	Linear time, but state count can explode in memory	grep -E (in some implementations), lexers
RE2 lazy DFA / automaton	Builds DFA states on demand, caches them, falls back safely if the cache overflows	Linear time, bounded memory — this is what SQL Server 2025 uses	RE2 — Google; also used in Go's regexp package

The practical takeaway for this session: SQL Server's regex cost model looks like "scan the string once," not "pattern complexity times string length, worst case."

What Each Kind of Operation Actually Costs

Operation	Question answered	SQL Server function	Cost driver
Match / test	Does the pattern match? (boolean)	REGEXP_LIKE	Full pattern evaluation per row; no index seek
Search / find	Where does it match? (position)	REGEXP_INSTR	Similar to match, plus position bookkeeping
Extract / capture	What did it capture? (groups)	REGEXP_SUBSTR, REGEXP_MATCHES	Group tracking adds overhead over a plain test
Replace	Substitute matches	REGEXP_REPLACE	Allocation cost of the new string matters at scale
Split	Tokenize on a delimiter pattern	REGEXP_SPLIT_TO_TABLE	Row-generation cost scales with token count
Global / all matches	Find every match in a long string	REGEXP_MATCHES	Cost scales with match count, not just string length

How REGEXP_* Compares to Legacy T-SQL

- *These figures are from independent testing (Redgate/Simple-Talk, early 2026) — always validate against your own data and hardware*
- **REGEXP_LIKE vs. LIKE**
  - LIKE can be 10–50x faster on large tables when an equivalent wildcard pattern exists — because LIKE can seek an index and REGEXP_LIKE cannot
- **REGEXP_REPLACE vs. legacy PATINDEX + SUBSTRING + REPLACE**
  - REGEXP_REPLACE measured roughly 5x faster — the standout performance win among the seven functions
- **REGEXP_COUNT, REGEXP_SUBSTR, REGEXP_INSTR**
  - Roughly on par with their best legacy equivalents (CLR-based counting, PATINDEX + SUBSTRING) — REGEXP_COUNT is ~5x faster than a plain T-SQL UDF, if that's your baseline
- **REGEXP_SPLIT_TO_TABLE**
  - Close to STRING_SPLIT and CLR-based splitting — a wash on speed, a win on flexibility

REAL-WORLD USE CASES

Where Regex Earns Its Keep

Use case	Function(s)	Why regex beats the legacy alternative
Validate emails, phone numbers on insert	REGEXP_LIKE, CHECK constraint	One expression instead of chained LIKE/PATINDEX rules
Redact PII before sending to a reporting layer	REGEXP_REPLACE	Single call replaces PATINDEX + SUBSTRING + REPLACE
Parse deadlock graphs / blocked-process reports	REGEXP_SUBSTR, REGEXP_MATCHES	Pulls database, object, and RID out of free-form XML/text reliably
Tokenize free-text log lines	REGEXP_SPLIT_TO_TABLE	Splits on a pattern, not just a fixed delimiter
Import mixed-format legacy data	REGEXP_MATCHES	Table output with JSON capture groups — good fit for staged loads

Best candidates are new queries and messy/legacy data — not a reason to refactor working legacy-function code (see next slide).

Can you Index a Regex predicate?

Short answer: no index understands RE2 patterns — REGEXP_LIKE is always a residual filter.

- **No B-tree, full-text, or columnstore index speaks RE2.** The engine has no data structure built for arbitrary regex patterns — every REGEXP_LIKE call scans and tests each row's text.
- **The optimizer bolts it on as a Filter.** Whatever access path it already picked — seek, scan, index intersection — REGEXP_LIKE runs afterward, on the rows that survive.
- **What actually matters: how many rows reach the filter.** An index still pays off if it narrows the input before the pattern runs — it just can't drive the regex match itself.
- **Put sargable predicates first.** Let =, BETWEEN, or LIKE 'prefix%' do the heavy lifting in the WHERE clause; use REGEXP_LIKE to refine what's left, not to scan the whole table.

Making Regex Indexable

The workound: compute the macth once, persist it, then index the results - no patterns.

1. EXTRACT INTO A PERSISTED COLUMN

```
ALTER TABLE dbo.Orders
  ADD OrderCode AS
    REGEXP_SUBSTR(Notes,
      'ORD-\d{4}-\d{6}') PERSISTED;

CREATE INDEX IX_OrderCode
  ON dbo.Orders(OrderCode);

-- this now SEEKS:
WHERE OrderCode = 'ORD-2025-000123';
```

2. FLAG + FILTERED INDEX

```
ALTER TABLE dbo.BenchRows
  ADD IsValidEmail AS
    CASE WHEN REGEXP_LIKE(EmailCol,
      '^[A-Za-z0-9._%+-]+@...$')
    THEN 1 ELSE 0 END PERSISTED;

CREATE INDEX IX_ValidEmail
  ON dbo.BenchRows(RowID)
  WHERE IsValidEmail = 1;
```

Caviat

The regex runs once per row, at write time — not once per query. Reads become index seeks instead of full scans.

The cost moves to INSERT/UPDATE. Worth it for read-heavy tables; think twice on a high-frequency OLTP write path.

PRACTICAL GUIDANCE

Rules of Thumb Before You Ship It

- If a simple LIKE or PATINDEX meets the need, use that — regex adds power you may not need and complexity you'll have to test
- Avoid REGEXP_* functions in WHERE clauses and JOINs when you have any alternative; filter first, then apply regex to the smaller set
- Benchmark every pattern you ship — a single added comparison can noticeably change execution time; don't assume one query's numbers generalize to another
- Be extra deliberate with Unicode (nvarchar) patterns — this is where the biggest legacy-vs-regex performance gaps show up
- Regex patterns are precise by nature — case sensitivity defaults to on (flag 'c'), unlike some legacy comparisons; test edge cases explicitly
- Good new-code candidates: complex extraction/replace logic, teams with an existing regex library, environments where deploying CLR assemblies isn't allowed
- Not a good reason to refactor: existing, working legacy-function code with no functional gap to close

Patterns That Bite — And Why RE2 Doesn't Get Bitten

- **Classic catastrophic-backtracking patterns (on a backtracking engine, NOT SQL Server 2025):**
  - (a+)+$ — nested quantifiers over the same character create exponentially many ways to fail
  - (a|a)*b — ambiguous alternation lets the engine retry the same ground repeatedly
  - (.*)(.*)(.*)= — greedy groups re-scanning overlapping territory
  - Feed any of these a long run of 'a's with no trailing match, on .NET/PCRE/Java/JS, and CPU time explodes
- **On SQL Server 2025's RE2 engine, all three still run in linear time — this is the practical payoff of everything on the last few slides**
- ***A cultural aside, not a regex pattern:***
  - :(){:|:&};: is the classic Bash "fork bomb" — a shell function that recursively re-launches itself until the OS runs out of processes. It's often filed alongside regex death-traps because it's the same genre of joke (a few characters, total chaos) — but it's shell syntax, not a regular expression, and SQL Server's regex engine has nothing to do with it

Key Takeaways

- **Seven functions, two shapes: five scalar (LIKE, COUNT, INSTR, REPLACE, SUBSTR) for SELECT/WHERE, two table-valued (MATCHES, SPLIT_TO_TABLE) for FROM**
- Compatibility level 170 gates three of the seven — check it before you demo this live
- The engine is RE2: linear-time guaranteed, no ReDoS — at the cost of no back-references and no lookaround
- No REGEXP_* function seeks an index today; filter first, apply regex to the smallest set you can
- REGEXP_REPLACE is the clearest performance win of the seven; REGEXP_LIKE is the one to be most careful with in WHERE clauses
- Best used in new code with a real complexity need — not a wholesale replacement for LIKE, PATINDEX, or working CLR code

Thank you & Q&A


http://tomaztsql.wordpress.com


tomaz.kastrun@gmail.com

@tomaz_tsql


/in/tomaztsql


http://github.com/tomaztk



https://mvp.microsoft.com/PublicProfile/5002196

Resources

- Microsoft Learn — Regular Expressions Functions (Transact-SQL): learn.microsoft.com/sql/t-sql/functions/regular-expressions-functions-transact-sql
- Microsoft Learn — Regular Expressions overview: learn.microsoft.com/sql/relational-databases/regular-expressions/overview
- Azure SQL Dev Blog — GA announcement for regex support in SQL Server 2025
- Simple-Talk / Redgate — "SQL Server Regular Expression Performance and Guidelines" (source for the benchmark figures in this deck

Do you feel like writing RegEx?

NLB is hiring!



- PostgreSQL / Oracle DBA (m/ž)
- SRE / DevOps inženir
- Backend Java razvijalec v organizacijski enoti IT Razvoj
- Samostojni analitik informacijskih sistemov

WE APPRECIATE YOUR FEEDBACK,
AND IT MAKES US BETTER EACH YEAR.

YOU CAN COLLECT ADDITIONAL
NT COINS BY FILLING THE QUESTOINEERS.

THEN VISIT GEEK BOUTIQUE AND
SPEND NT COINS ON COOL GIVEAWAYS.

